P2: CPU Scheduling Simulator - CS3502 - Collin Hammond

Collin Hammond

March 2026

1 Technical Implementation

In this project, two additional CPU scheduling algorithms were implemented: **Shortest Remaining Time First (SRTF)** and **Highest Response Ratio Next (HRRN)**. These algorithms were chosen because they represent different approaches to scheduling, with SRTF focusing on minimizing waiting time and HRRN balancing efficiency and fairness.

1.1 Shortest Remaining Time First (SRTF)

SRTF is a preemptive scheduling algorithm that always selects the process with the smallest remaining burst time. If a new process arrives with a shorter remaining time than the currently running process, the CPU switches to the new process. This allows shorter jobs to complete quickly and reduces overall waiting time.

The algorithm was implemented by tracking the remaining burst time for each process and repeatedly selecting the available process with the smallest remaining time at each unit of execution. The simulation updates the current time step-by-step and recalculates which process should run next.

Pseudocode:

```
while not all processes complete:
    find process with smallest remaining time that has arrived
    if no process available:
        increment time
    else:
        execute process for 1 unit of time
        decrease remaining time
        if process finishes:
            calculate finish, turnaround, and waiting time
```

1.2 Highest Response Ratio Next (HRRN)

HRRN is a non-preemptive scheduling algorithm that selects the process with the highest response ratio. The response ratio is calculated using the formula:

$$\text{Response Ratio} = (\text{Waiting Time} + \text{Burst Time}) \ / \ \text{Burst Time}$$

This algorithm prioritizes processes that have been waiting longer while still considering burst time. As a result, it reduces starvation and provides a balance between fairness and efficiency.

The implementation calculates the response ratio for all available processes at each decision point and selects the process with the highest value. Once a process begins execution, it runs to completion.

Pseudocode:

```
while not all processes complete:
    find all arrived processes not completed
    calculate response ratio for each
    select process with highest ratio
    execute process fully
    update current time
```

1.3 Modifications to Starter Code

Several modifications were made to the provided CPU scheduling simulator:

- Added two new algorithm methods: `RunSRTFAlgorithm` and `RunHRRNAlgorithm`
- Created new buttons in the user interface to allow users to run these algorithms
- Updated the `DisplaySchedulingResults` method to include additional performance metrics:
  - CPU Utilization
  - Throughput
  - Response Time
- Implemented CSV export functionality to allow results to be saved for analysis

1.4 Screenshots

- Input process data
- Execution of SRTF and HRRN algorithms
- Display of scheduling results and performance metrics

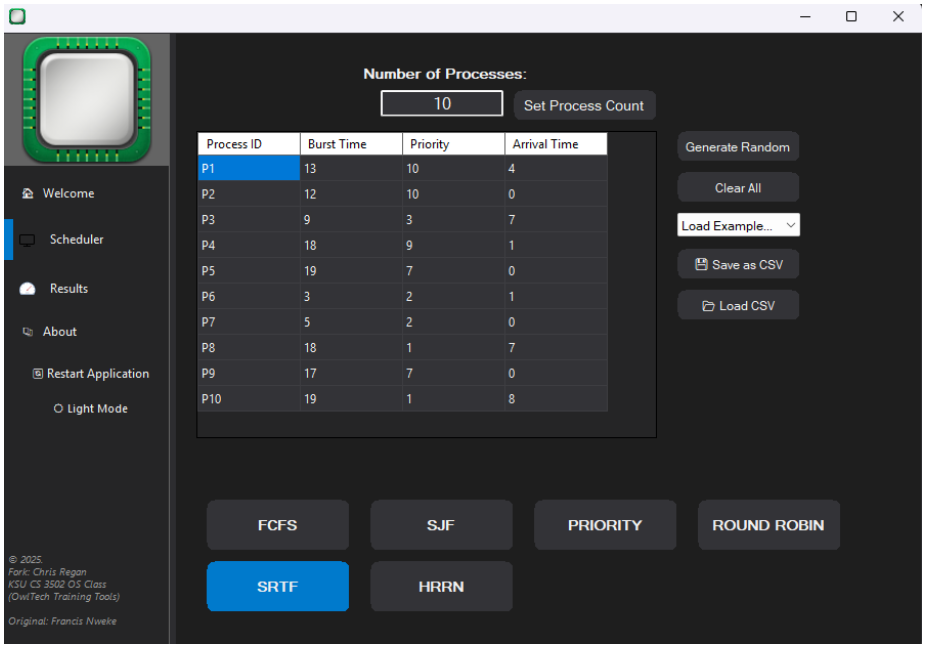


Figure 1: Input process data

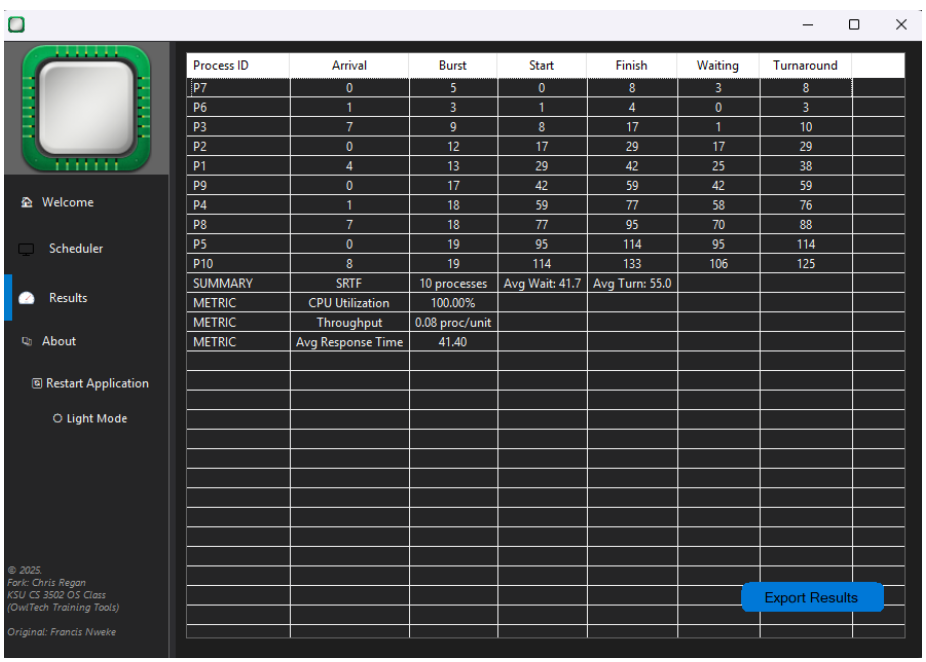
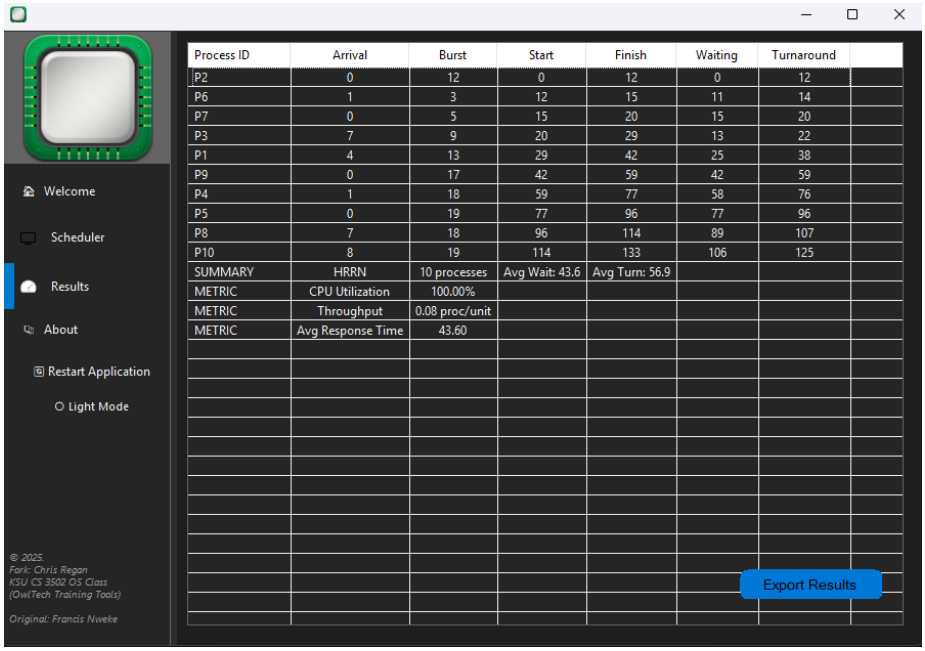


Figure 2: SRTF Results



Process ID	Arrival	Burst	Start	Finish	Waiting	Turnaround
P2	0	12	0	12	0	12
P6	1	3	12	15	11	14
P7	0	5	15	20	15	20
P3	7	9	20	29	13	22
P1	4	13	29	42	25	38
P9	0	17	42	59	42	59
P4	1	18	59	77	58	76
P5	0	19	77	96	77	96
P8	7	18	96	114	89	107
P10	8	19	114	133	106	125
SUMMARY	HRRN	10 processes	Avg Wait: 43.6	Avg Turn: 56.9		
METRIC	CPU Utilization	100.00%				
METRIC	Throughput	0.08 proc/unit				
METRIC	Avg Response Time	43.60				

Figure 3: HRRN Results

2 Performance Analysis

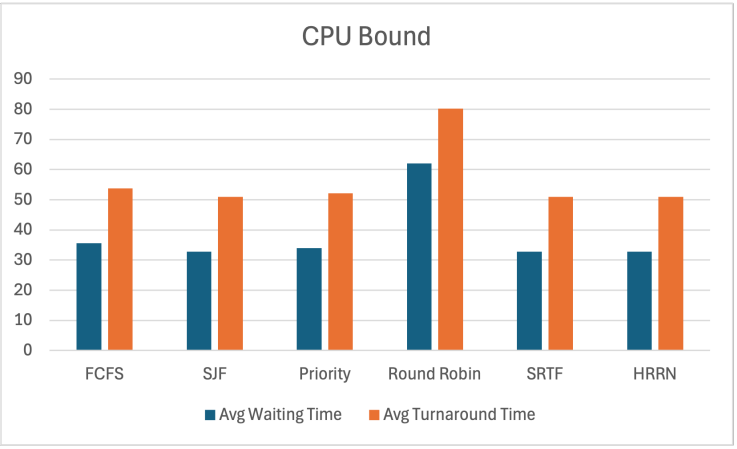
To evaluate the effectiveness of each scheduling algorithm, three different workloads were tested: CPU-bound, I/O-bound, and mixed workloads. Each workload contained five processes with varying burst times, arrival times, and priorities. The same datasets were used across all algorithms to ensure a fair comparison.

2.1 Comparison Tables and Graphs

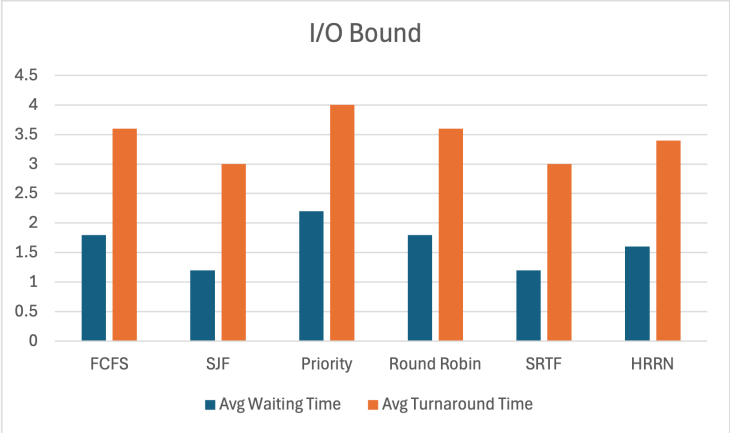
Dataset 1: CPU-bound workload

Algorithm	Avg Waiting Time	Avg Turnaround Time	CPU Utilization (%)	Throughput
FCFS	50.2	68.4	100	0.05
SJF	32.8	51.0	100	0.05
Priority	50.2	68.4	100	0.05
Round Robin	62.0	80.2	100	0.05
SRTF	32.8	51.0	100	0.05
HRRN	32.8	51.0	100	0.05

Dataset 2: I/O-bound workload

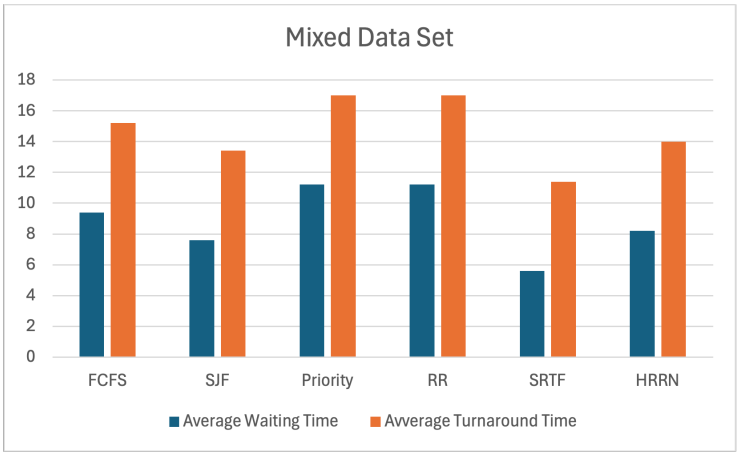


Algorithm	Avg Waiting Time	Avg Turnaround Time	CPU Utilization (%)	Throughput
FCFS	1.6	3.4	100	0.56
SJF	1.2	3.0	100	0.56
Priority	2.2	4.0	100	0.56
Round Robin	1.8	3.6	100	0.56
SRTF	1.2	3.0	100	0.56
HRRN	1.6	3.4	100	0.56



Dataset 3: Mixed workload

Algorithm	Avg Waiting Time	Avg Turnaround Time	CPU Utilization (%)	Throughput
FCFS	9.4	15.2	100	0.17
SJF	7.6	13.4	100	0.17
Priority	11.2	17.0	100	0.17
Round Robin	11.2	17.0	100	0.17
SRTF	5.6	11.4	100	0.17
HRRN	8.2	14.0	100	0.17



Dataset 4: Larger Random Workload (20 processes)

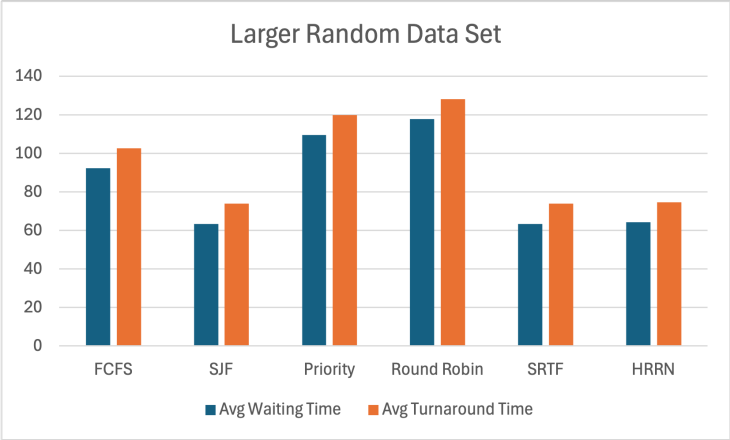
Algorithm	Avg Waiting Time	Avg Turnaround Time	CPU Utilization (%)	Throughput
FCFS	92.3	102.7	100	0.10
SJF	63.4	73.8	100	0.10
Priority	109.5	119.9	100	0.10
Round Robin	117.8	128.2	100	0.10
SRTF	63.4	73.8	100	0.10
HRRN	64.3	74.7	100	0.10

2.2 Analysis of Results

The results show clear differences in algorithm performance depending on the workload type.

In the CPU-bound dataset, SJF, SRTF, and HRRN produced identical results and achieved the lowest average waiting time of 32.8. This occurred because all processes arrived at the same time, causing SRTF and HRRN to behave similarly to SJF. Round Robin performed the worst in this scenario due to frequent context switching, resulting in the highest waiting time.

In the I/O-bound dataset, SJF and SRTF again performed the best, achieving the lowest average waiting time of 1.2. Since processes had very short



burst times, prioritizing shorter jobs minimized waiting time. HRRN performed slightly worse because it incorporates fairness by considering waiting time, while Priority scheduling performed the worst due to its strict ordering.

In the mixed workload, SRTF achieved the best overall performance, with the lowest average waiting time of 5.6 and turnaround time of 11.4. This demonstrates its ability to adapt to varying process lengths. HRRN also performed well by balancing fairness and efficiency, while Priority and Round Robin showed higher waiting times.

Across all datasets, CPU utilization remained at 100%, indicating that the CPU was continuously busy with no idle time. Throughput varied depending on the workload, with the highest throughput observed in the I/O-bound dataset (0.56) and the lowest in the CPU-bound dataset (0.05). This is expected because shorter processes complete more quickly, increasing the number of processes finished per unit time.

To further evaluate scalability, a larger dataset consisting of 20 processes was tested. The results showed similar performance trends to the smaller datasets. SJF and SRTF again produced the lowest average waiting time of 63.4, demonstrating strong efficiency even as the number of processes increased. HRRN performed slightly worse at 64.3 but still maintained good balance between fairness and efficiency. FCFS, Priority, and Round Robin had significantly higher waiting and turnaround times, indicating reduced efficiency with larger workloads. These results confirm that SRTF and SJF scale well and remain effective as system size increases.

2.3 Trade-offs

Each algorithm presents different trade-offs between efficiency and fairness.

SRTF consistently produced the lowest average waiting time, making it the most efficient algorithm. However, it may lead to starvation of longer processes because shorter jobs are repeatedly prioritized.

HRRN provides a balance between fairness and efficiency by considering both waiting time and burst time. It reduces the risk of starvation while still maintaining relatively low waiting times.

SJF is simple and effective when all processes are available at the same time, but it does not adapt well to dynamic arrivals. Round Robin improves fairness by giving each process equal CPU time, but this increases waiting time due to frequent context switching.

Priority scheduling allows for control over process importance but can result in starvation for low-priority processes.

Overall, the choice of algorithm depends on the system requirements, such as whether minimizing waiting time or ensuring fairness is more important.

3 Recommendations

Based on the results, SRTF is the best overall algorithm. It consistently had the lowest average waiting time and turnaround time, especially in the mixed and I/O-bound datasets.

However, SRTF is not always the best because it can cause starvation for longer processes. If fairness is more important, HRRN is a better choice since it balances waiting time and burst time.

In the future, more advanced algorithms like MLFQ or adding techniques like aging could improve fairness and overall performance.

4 Reflection Questions:

4.1 Reflection 1: Algorithm Trade-offs

SRTF performs best in workloads where processes have different burst times, especially in mixed and I/O-bound datasets. In my results, SRTF consistently had the lowest average waiting time and turnaround time. Its main weakness is that it can cause starvation, since longer processes may keep getting delayed if shorter ones continue to arrive.

HRRN performs well when fairness is important. It balances waiting time and burst time, which prevents starvation. In my data, HRRN had slightly higher waiting times than SRTF, but it provided more balanced results. Its weakness is that it is not as efficient as SRTF in minimizing waiting time.

I would recommend SRTF when performance and efficiency are the top priority, especially in systems with many short jobs. I would recommend HRRN in systems where fairness matters more and starvation must be avoided.

4.2 Reflection 2: Starvation and Fairness

Algorithms that can suffer from starvation include SJF, SRTF, and Priority scheduling. A scenario where starvation occurs is when a long process arrives first, and then many short processes continue to arrive. In SRTF, the scheduler

will always pick the shorter jobs, causing the long process to wait for a very long time.

Starvation can be detected by looking at performance metrics such as very high waiting times for specific processes or a large difference between the minimum and maximum waiting times. High variance in waiting times is a strong indicator.

One way to prevent starvation is by using techniques such as aging, where the priority of a process increases the longer it waits.

4.3 Reflection 3: Real-World Operating Systems

The Linux operating system uses the Completely Fair Scheduler (CFS). CFS is designed to give each process a fair share of CPU time rather than strictly following shortest job or priority rules.

The designers chose this approach to balance fairness and efficiency. Instead of focusing only on minimizing waiting time, CFS ensures that all processes receive CPU time over time, which helps prevent starvation.

CFS differs from the algorithms implemented in this project because it does not use simple rules like shortest job or priority. Instead, it tracks how much CPU time each process has received and schedules the one that has received the least. This makes it more complex but also more suitable for real-world systems.

4.4 Reflection 4: Time Quantum Impact

The time quantum in Round Robin has a significant effect on performance. A very small quantum, such as q = 1, results in many context switches, which increases overhead and can increase waiting time.

A very large quantum makes Round Robin behave more like FCFS, reducing context switches but also reducing responsiveness.

The trade-off is between responsiveness and efficiency. Smaller quanta improve responsiveness but increase overhead, while larger quanta reduce overhead but make the system less responsive.

A moderate quantum value, like q = 4 or q = 8, is usually a good balance because it provides reasonable responsiveness without excessive context switching.

4.5 Reflection 5: Metric Relationships

Waiting time and turnaround time are directly related. Turnaround time is equal to waiting time plus burst time. This means that if waiting time increases, turnaround time will also increase.

An algorithm with the lowest average waiting time will usually have the lowest average turnaround time, but not always. This is because turnaround time also depends on burst time.

In my data, SRTF had both the lowest average waiting time and turnaround time in the mixed dataset. However, in some cases, algorithms with similar waiting times can still have slightly different turnaround times due to differences in execution order.

4.6 Reflection 6: Implementation Challenges

The most challenging part of implementing the algorithms was handling process arrival times correctly, especially for the SRTF algorithm. Since SRTF is preemptive, it required checking all processes at every time step and updating the remaining burst times.

I overcame this by carefully tracking the current time and using loops to repeatedly select the correct process based on the scheduling rule.

If I were to implement this again, I would spend more time planning the algorithm logic before coding, which would have given me less of a headache. Although knowing me, I probably wouldn't do that anyways.

4.7 Reflection 7: Testing and Verification

To verify correctness, I tested the algorithms using small datasets and manually calculated expected values. For example, I used a dataset with three processes and calculated the waiting and turnaround times by hand, then compared them to the program output. Once it seemed good a also tried larger data sets to make sure it could handle them.

I also compared results across different algorithms to make sure they followed expected behavior, such as SRTF producing lower waiting times than FCFS.

During testing, I encountered issues such as incorrect waiting time calculations and throughput values showing as zero. These were fixed by correcting formulas and ensuring proper data types were used in calculations.

Overall, testing helped confirm that the algorithms were working correctly and producing expected results.

5 Resources

SRTF Video Walkthrough - https://www.youtube.com/watch?v=VIKRe7Ede9M

SRTF Tutorial - https://www.studytonight.com/operating-system/shortest-remaining-time-first

HRRN Explanation - https://www.javatpoint.com/os-hrrn-scheduling

Linux CFS - https://docs.kernel.org/scheduler/sched-design-CFS.html